SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT-5

1. Student Details

StudentName	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A-2
Roll No.	202501110069
Date ofSubmission	20-10-2006

2. Practical Details

Practical No.	5
Practical Title	Classes and Objects, this Pointer, Inline Function, Static Member Function and Friend Class in C++
Date Performed	
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

(a)Writea C++ programthat defines a class with private data members, a constructor and member functions, creates objects of the class, and uses the **this** pointer (to resolve name conflicts, to return the calling object for function chaining and to display the address of the current object). Also use **inline** member functions.

(b) Extend the program to use a **static data member** and a **static member function** to count the number of objects, and a **friend class** that can access the private members of another class.

4. Objectives

1. To understand the concept of a class and an object and how data and functions are bound together.
2. To understand the working of the this pointer and its uses.
3. To use inline functions and understand their advantages and limitations.
4. To understand static data members and static member functions.
5. To understand friend functions and friend classes and how they access private members.

5. Expected Learning Outcomes

- Declare aclass,create objectsandaccess members using the dot operator.
- Use the this pointer for member access, function chaining and object identification.
- Write inline member functions and decide when inline is suitable.
- Share data among all objects using static members and call static functions with the class name.
- Grant controlled access to private data using a friend class.

6. Theory / Concept

Classand Object: A class is a user-defined data type that groups data members (variables) and member functions (methods) into a single unit. It is a blueprint and does not occupy memory by itself. An object is an instance of a class; memory is allocated for the data members when an object is created. Members

declared private are accessible only inside the class, while public members are accessible from outside. This is called encapsulation and data hiding. A constructor is a special member function with the same name as the class that is called automatically when an object is created, and a destructor (~ClassName) is called automatically when the object is destroyed.

this pointer: Every non-static member function receives a hidden pointer named **this** that holds the address of the object which called the function. It is used (i) to differentiate between data members and parameters having the same name (this->marks = marks;), (ii) to return the calling object (return *this;) so that function calls can be chained (s.setName("A").setMarks(50);), and (iii) to get the address of the current object. The this pointer is a constant pointer and cannot be reassigned. It is not available in static member functions.

Inline function: An inline function is a request to the compiler to replace the function call with the body of the function at the point of the call, which removes the overhead of a normal function call (stack frame creation, jump and return). A function defined inside the class body is implicitly inline; a function defined outside can be made inline by using the inline keyword. Inline is best for small, frequently called functions. It increases code size if a large function is inlined, and the compiler may ignore the request for functions containing loops, recursion or static variables.

Static data member and static member function: A static data member is declared with the keyword static. Only one copy of it exists and it is shared by all objects of the class. It is declared inside the class and must be defined (and initialized) outside the class using the scope resolution operator, for example int Student::count = 0;. A static member function belongs to the class and not to any object. It can access only static members, has no this pointer, and can be called using the class name, for example `Student::getCount();`.

Friend function and friend class: A friend is declared using the keyword friend and is allowed to access the private and protected members of the class even though it is not a member of that class. If a class is declared as a friend (friend class Auditor;), all member functions of the friend class can access the private members of the other class. Friendship is not mutual (if A is a friend of B, B is not automatically a friend of A), it is not inherited and it is not transitive. Friend breaks encapsulation to some extent, so it should be used carefully.

7. Algorithm / Pseudocode

1. START
2. Declare a forward declaration of class Auditor.
3. Define class Student with private members rollNo, name, marks and a static member count; declare Auditor as a friend class.
4. Write a constructor using this->rollNo = rollNo, etc. and increase count; write a destructor that decreases count.
5. Write setName() and setMarks() that return *this, inline functions getMarks() and isPass(), getAddress() that returns this, display() and the static function getCount().
6. In main(), read roll number, name and marks and create object s1; display it and check that s1.getAddress() == &s1.
7. Create object s2 and call s2.setName("Aarav").setMarks(35) using function chaining; display s2.
8. Call the inline function isPass() for s1 and s2 and display Pass or Fail.
9. Display Student::getCount(); create a temporary object inside a block and display the count again inside and after the block.
10. Create an object of Auditor; call showPrivate(s2) and addGraceMarks(s2, 10) and display s2 again.
11. STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose

Programming Language	C++
IDE / Platform	Visual Studio Code (VS Code)
Libraries / Packages	iostream, string

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	
Repository Name	Assignment5
Branch Used	main
Commit ID / Latest Commit	Add file via upload
Date of Last Commit	20-10-2006

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
1	46 Riyansh 72.5	this == &s1; s1: Roll 46, Homesh, 72.5 -> Pass; s2 (chained) Aarav, 35 -> Fail; count = 2, then 3 inside block, then 2; after grace marks s2 = 45 -> Pass	same as expected	pass
2	7 Sneha 30	this == &s1; s1: Roll 7, Sneha, 30 -> Fail; s2 (chained) Aarav, 35 -> Fail; count = 2, then 3 inside block, then 2; after grace marks s2 = 45 -> Pass	same as expected	pass

```
--- Part A: Class, Object and this Pointer ---
Enter roll no, name and marks of student 1: 202 Homesh 23
Roll No: 202, Name: Homesh, Marks: 23
Address of s1     = 0x61fe80
Value of this     = 0x61fe80
this pointer points to the calling object
Roll No: 0, Name: Aarav, Marks: 35

--- Part B: Inline and Static Member Function ---
s1 result: Fail
s2 result: Fail
Total students = 2
Inside block, total students = 3
After block, total students = 2

--- Part C: Friend Class ---
Auditor reads private data -> Roll No: 0, Name: Aarav, Marks: 35
After adding grace marks: Roll No: 0, Name: Aarav, Marks: 45
s2 result: Pass
```

11. Results / Observations

The C++ program was implemented in Visual Studio Code. In Part A, the object s1 was created from user input and the this pointer gave the same address as &s1, which shows that this holds the address of the calling object. Function chaining on s2 worked because setName() and setMarks() return *this. In Part B, the inline function isPass() gave the correct result for each object, and the static data member count was shared by all objects: it increased when the temporary object was created and decreased when its destructor ran at the end of the block. getCount() was called with the class name without any object. In Part C, the friend class Auditor accessed the private members of Student directly and modified the marks of s2. Both test cases gave the expected output.

12. Conclusion

The practical successfully demonstrated classes and objects, the this pointer, inline functions, static data members with static member functions, and friend classes in C++. It showed how a class encapsulates data and functions, how this identifies the current object, how static members are shared across all objects, and how a friend class can be given controlled access to private data when it is really required.

13. Assignment Questions

Q1. What is the difference between a class and an object? What is the this pointer?

Answer: A class is a blueprint or user-defined data type that describes the data members and member functions; it does not take memory until an object is created. An object is an instance of a class and has its own copy of the non-static data members. Many objects can be created from one class. The this pointer is a hidden constant pointer passed to every non-static member function that stores the address of the calling object. It is used to resolve the name conflict between data members and parameters, to return the current object (return *this) for chaining of calls, and to get the address of the object. It cannot be used inside a static member function.

Q2. What is an inline function? State its advantages and limitations.

Answer: An inline function is a function for which the compiler replaces the call with the function body. A member function defined inside the class is inline by default, or the inline keyword can be used. Advantages: it avoids the overhead of a function call and return, it is faster for small functions and it is type safe, unlike macros. Limitations: it increases the size of the executable if the function is large or called many times, it is only a request and the compiler may ignore it for functions with loops, recursion or static variables, and changing an inline function requires recompiling all the code that uses it.

Q3. Explain static data members and static member functions.

Answer: A static data member is shared by all objects of the class and only one copy exists in memory, independent of the number of objects. It is declared inside the class and defined outside it with the scope resolution operator. A static member function belongs to the class, can be called as ClassName::function() without creating an object, can access only static data members and other static functions, and does not have a this pointer. A common use is counting the number of objects created, as done in this practical.

Q4. What is a friend class? Write its advantages and disadvantages.

Answer: A friend class is a class declared with the friend keyword inside another class, so that all its member functions can access the private and protected members of that class. It is useful when two classes are closely related and need to work together, for example an Auditor class that needs to read or change the data of a Student class. Friendship is not mutual, not inherited and not transitive. Its disadvantage is that it breaks data hiding and encapsulation and increases coupling between the classes, so it should be used only when necessary.

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S